

Realtime Chat: Flutter + Firebase

Implementation guide for Cursor — uses your existing components and state management

1. Dependencies (pubspec.yaml)

```
dependencies:
  firebase_core: ^3.0.0
  firebase_auth: ^5.0.0
  cloud_firestore: ^5.0.0
  uuid: ^4.0.0
```

2. Firestore Data Model

Two collections: a top-level **chats** collection and a **messages** subcollection per chat.

```
/chats/{chatId}
members:      [uid1, uid2]          # for querying chats by user
memberNames:  {uid: displayName}    # avoids extra fetches in the list
lastMessage:  "hey!"
lastMessageAt: Timestamp
unreadCount:  {uid1: 0, uid2: 2}

/chats/{chatId}/messages/{messageId}
senderId:     "uid123"
senderName:   "Alice"
text:         "Hello!"
createdAt:    FieldValue.serverTimestamp()
readBy:       [uid1]                # for read receipts
```

Use a subcollection for messages — never an array on the chat doc. Arrays cannot be paginated, queried, or ordered, and Firestore documents have a 1 MB size limit.

3. Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /chats/{chatId} {
      allow read, update: if request.auth != null
        && request.auth.uid in resource.data.members;

      allow create: if request.auth != null
        && request.auth.uid in request.resource.data.members;

    }

    match /messages/{messageId} {
      allow read: if request.auth != null
        && request.auth.uid in get(
          /databases/{database}/documents/chats/{chatId}
        ).data.members;
    }
  }
}
```

```

        allow create: if request.auth != null
            && request.auth.uid in get(
                /databases/$(database)/documents/chats/$(chatId)
            ).data.members
            && request.resource.data.senderId == request.auth.uid;
    }
}
}
}
}

```

4. ChatService

A single service class that encapsulates all Firestore reads and writes. Wire this into your existing state management (Riverpod / BLoC / Provider).

4a. Create or fetch a 1:1 chat

```

// Deterministic chatId – same two users always resolve to the same doc
Future<String> getOrCreateChat(String otherUid, String otherName) async {
    final ids = [currentUid, otherUid]..sort();
    final chatId = ids.join('_');
    final ref = _firestore.collection('chats').doc(chatId);

    if (!(await ref.get()).exists) {
        await ref.set({
            'members': ids,
            'memberNames': {currentUid: currentName, otherUid: otherName},
            'lastMessage': '',
            'lastMessageAt': FieldValue.serverTimestamp(),
            'unreadCount': {currentUid: 0, otherUid: 0},
        });
    }
    return chatId;
}

```

4b. Send a message (batched write)

```

Future<void> sendMessage(String chatId, String text) async {
    final trimmed = text.trim();
    if (trimmed.isEmpty) return;

    final batch = _firestore.batch();

    // 1. Add message to subcollection
    final msgRef = _firestore
        .collection('chats').doc(chatId)
        .collection('messages').doc(const Uuid().v4());
    batch.set(msgRef, {
        'senderId': currentUid,
        'senderName': currentName,
        'text': trimmed,
        'createdAt': FieldValue.serverTimestamp(),
        'readBy': [currentUid],
    });

    // 2. Update chat metadata atomically
}

```

```

final chatRef = _firestore.collection('chats').doc(chatId);
batch.update(chatRef, {
  'lastMessage':      trimmed,
  'lastMessageAt':    FieldValue.serverTimestamp(),
  'unreadCount.$currentUid': 0,
});

await batch.commit();
}

```

4c. Realtime streams

```

// Messages stream – drives the chat screen
Stream<List<Message>> messagesStream(String chatId) => {
  _firestore
    .collection('chats').doc(chatId).collection('messages')
    .orderBy('createdAt', descending: false)
    .snapshots()
    .map((s) => s.docs.map(Message.fromDoc).toList());

// Chat list stream – drives the inbox/conversation list
Stream<List<ChatPreview>> chatsStream() => {
  _firestore
    .collection('chats')
    .where('members', arrayContains: currentUid)
    .orderBy('lastMessageAt', descending: true)
    .snapshots()
    .map((s) => s.docs
      .map((d) => ChatPreview.fromDoc(d, currentUid))
      .toList());

// Mark as read
Future<void> markAsRead(String chatId) => {
  _firestore.collection('chats').doc(chatId).update({
    'unreadCount.$currentUid': 0,
  });
}

```

5. Models

Message

```

class Message {
  final String id, senderId, senderName, text;
  final DateTime? createdAt;
  final List<String> readBy;

  factory Message.fromDoc(DocumentSnapshot doc) {
    final d = doc.data() as Map<String, dynamic>;
    return Message(
      id:      doc.id,
      senderId: d['senderId'] ?? '',
      senderName: d['senderName'] ?? '',
      text:      d['text'] ?? '',
      createdAt: (d['createdAt'] as Timestamp?).toDate(),
      readBy:      List<String>.from(d['readBy'] ?? []),
    );
  }
}

```

```

    bool get isFromCurrentUser =>
        senderId == FirebaseAuth.instance.currentUser?.uid;
}

```

ChatPreview

```

class ChatPreview {
    final String id, otherUserId, otherUserName, lastMessage;
    final DateTime? lastMessageAt;
    final int unreadCount;

    factory ChatPreview.fromDoc(DocumentSnapshot doc, String currentUid) {
        final d = doc.data() as Map<String, dynamic>;
        final members = List<String>.from(d['members']);
        final otherUid = members.firstWhere((id) => id != currentUid);
        final names = Map<String, dynamic>.from(d['memberNames'] ?? {});
        return ChatPreview(
            id: doc.id,
            otherUserId: otherUid,
            otherUserName: names[otherUid] ?? 'Unknown',
            lastMessage: d['lastMessage'] ?? '',
            lastMessageAt: (d['lastMessageAt'] as Timestamp?)?.toDate(),
            unreadCount: (d['unreadCount'] ?? {})[currentUid] ?? 0,
        );
    }
}

```

6. Chat Screen UI

```

class ChatScreen extends StatefulWidget {
    final String chatId, otherUserName;
    const ChatScreen({required this.chatId, required this.otherUserName, super.key});

    @override State<ChatScreen> createState() => _ChatScreenState();
}

class _ChatScreenState extends State<ChatScreen> {
    final _controller = TextEditingController();
    final _scrollController = ScrollController();
    final _chatService = ChatService();

    @override
    void initState() {
        super.initState();
        _chatService.markAsRead(widget.chatId);
    }

    void _send() {
        _chatService.sendMessage(widget.chatId, _controller.text);
        _controller.clear();
        _scrollToBottom();
    }

    void _scrollToBottom() {
        WidgetsBinding.instance.addPostFrameCallback((_) {
            if (_scrollController.hasClients) {

```

```

        _scrollController.animateTo(
          _scrollController.position.maxScrollExtent,
          duration: const Duration(milliseconds: 300),
          curve: Curves.easeOut,
        );
      }
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(widget.otherUserName)),
      body: Column(
        children: [
          Expanded(
            child: StreamBuilder<List<Message>>(
              stream: _chatService.messagesStream(widget.chatId),
              builder: (context, snapshot) {
                if (!snapshot.hasData)
                  return const Center(child: CircularProgressIndicator());
                WidgetsBinding.instance
                  .addPostFrameCallback((_) => _scrollToBottom());
                return ListView.builder(
                  controller: _scrollController,
                  padding: const EdgeInsets.symmetric(vertical: 8),
                  itemCount: snapshot.data!.length,
                  itemBuilder: (_, i) =>
                    MessageBubble(message: snapshot.data![i]),
                );
              },
            ),
          ),
          SafeArea(
            child: Padding(
              padding: const EdgeInsets.all(8),
              child: Row(
                children: [
                  Expanded(
                    child: TextField(
                      controller: _controller,
                      textCapitalization: TextCapitalization.sentences,
                      decoration: InputDecoration(
                        hintText: 'Message...',
                        border: OutlineInputBorder(
                          borderRadius: BorderRadius.circular(24),
                        ),
                        contentPadding: const EdgeInsets.symmetric(
                          horizontal: 16, vertical: 10),
                      ),
                    ),
                    onSubmitted: (_) => _send(),
                  ),
                  const SizedBox(width: 8),
                  IconButton.filled(onPressed: _send,
                    icon: const Icon(Icons.send)),
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }
}

```

```

    ],
  ),
),
],
),
);
}
}

```

MessageBubble widget

```

class MessageBubble extends StatelessWidget {
  final Message message;
  const MessageBubble({required this.message, super.key});

  @override
  Widget build(BuildContext context) {
    final isMe = message.isFromCurrentUser;
    final cs = Theme.of(context).colorScheme;
    return Align(
      alignment: isMe ? Alignment.centerRight : Alignment.centerLeft,
      child: Container(
        margin: EdgeInsets.only(
          top: 4, bottom: 4,
          left: isMe ? 64 : 16, right: isMe ? 16 : 64,
        ),
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
        decoration: BoxDecoration(
          color: isMe ? cs.primary : cs.surfaceVariant,
          borderRadius: BorderRadius.only(
            topLeft: const Radius.circular(18),
            topRight: const Radius.circular(18),
            bottomLeft: Radius.circular(isMe ? 18 : 4),
            bottomRight: Radius.circular(isMe ? 4 : 18),
          ),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(message.text,
              style: TextStyle(
                color: isMe ? cs.onPrimary : cs.onSurfaceVariant,
                fontSize: 15,
              ),
            ),
            if (message.createdAt != null)
              Padding(
                padding: const EdgeInsets.only(top: 4),
                child: Text(
                  _fmt(message.createdAt!),
                  style: TextStyle(
                    fontSize: 11,
                    color: (isMe ? cs.onPrimary : cs.onSurfaceVariant)
                      .withOpacity(0.6),
                  ),
                ),
              ),
          ],
        ),
      ),
    );
  }
}

```

```

        ],
      ),
    ),
  );
}

String _fmt(DateTime dt) =>
  '${dt.hour.toString().padLeft(2, '0')}:${dt.minute.toString().padLeft(2, '0')}';
}

```

7. Pagination (Long Histories)

Don't load all messages at once. Use **limitToLast** and fetch older pages on scroll-up.

```

// Initial load – last 50 messages
.orderBy('createdAt').limitToLast(50).snapshots()

// Load older messages (cursor pagination)
.orderBy('createdAt')
  .endBefore([oldestMessage.createdAt])
  .limitToLast(25)
  .get()

```

8. Key Gotchas for Cursor

- **serverTimestamp() is null on first snapshot.** The server hasn't responded yet. Always null-check *createdAt* before displaying it — the *MessageBubble* above already handles this.
- **Composite index required.** The *chatsStream* query (where + *orderBy*) needs a Firestore composite index. The first time you run the query, the error in the console includes a one-click link to create it.
- **Offline support is free.** Firestore caches writes locally and syncs when connectivity returns. No extra setup needed.
- **Wire ChatService into your state management.** If you use Riverpod, expose *ChatService* as a *Provider* and expose *messagesStream* / *chatsStream* as *StreamProviders*. For BLoC, emit new states from the stream listener.
- **Unread count.** The current implementation zeros out the sender's count on send, but does not increment the recipient's count. Add *FieldValue.increment(1)* on the recipient's field inside the *batch.update* call in *sendMessage*.